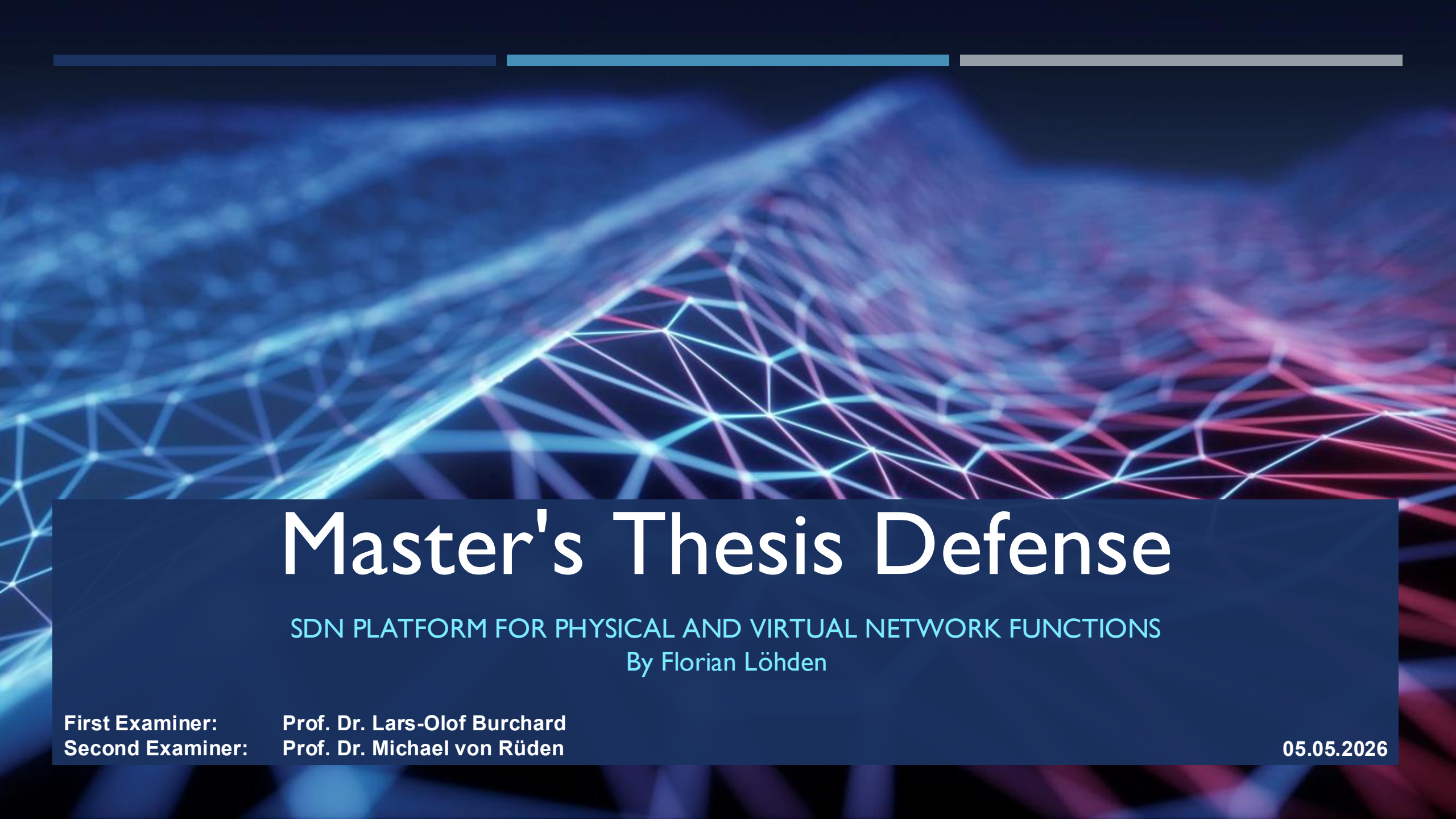




Master's Thesis Defense

SDN PLATFORM FOR PHYSICAL AND VIRTUAL NETWORK FUNCTIONS

By Florian Löhden

First Examiner:

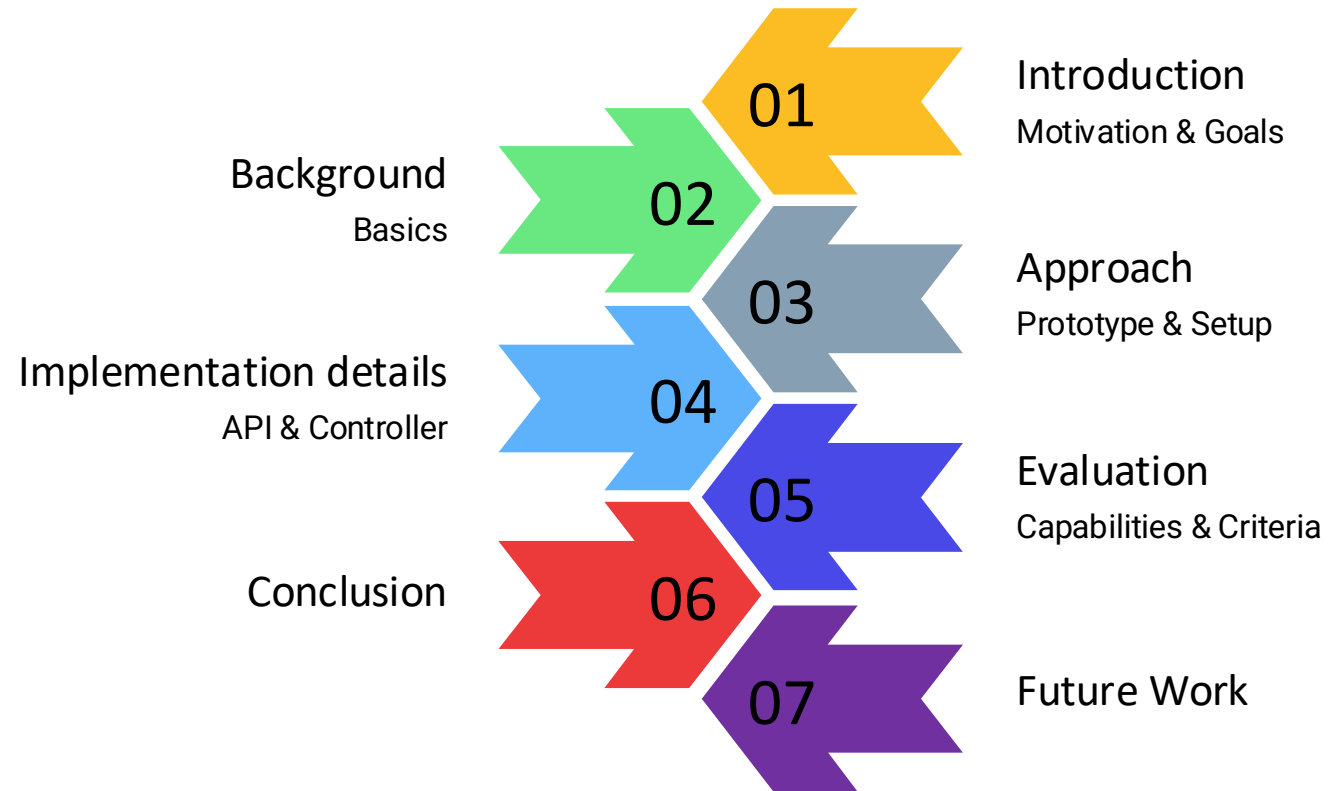
Prof. Dr. Lars-Olof Burchard

Second Examiner:

Prof. Dr. Michael von Rüden

05.05.2026

AGENDA



Motivation

Networks getting more complex

- More devices
- More functions
- More complex topologies

Expanding Requirements

- Protocols
- Security
- Performance Optimization

Containerization for applications

- **Usable for networking?**

Goals

Networks closer to Application teams

- End-to-end testing and troubleshooting

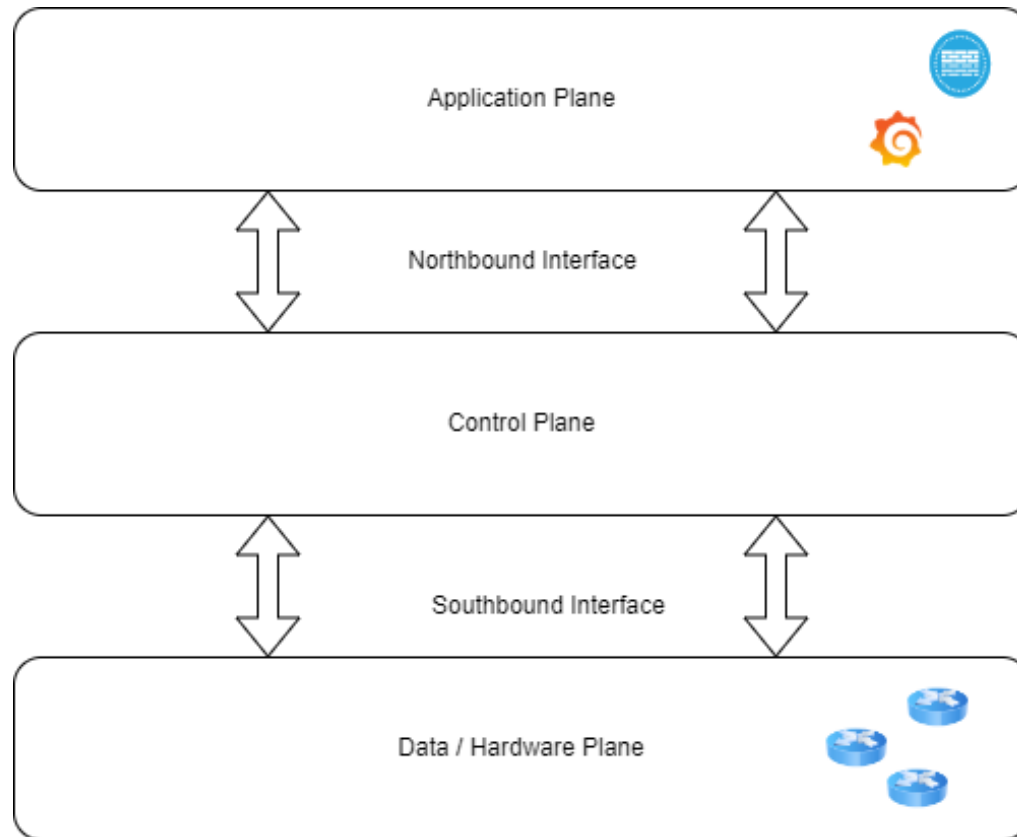
Managing Physical Devices with Kubernetes

- Managing VNF and PNF in one platform

Kubernetes Advantages in One Platform

- Declarative configuration, self healing and scalability

Basics - SDN



- **Application Plane**
 - Applications, Services, Policies
- **Control Plane**
 - Decision-making, Coordination, Adaptability
- **Data Plane**
 - Network devices

Basics – Kubernetes



Automation Engine

- Container Orchestration Platform
- High-Availability and Self-Healing
- Manages Virtual Network Functions



The Declarative Model

- Define the „Desired State“ via YAML
- Continuous reconciliation loop
- Platform automatically detects drifts



Extensibility

- Custom Resource Definitions
- The Operator Pattern
- Expanding API to new Domains

Basics – Protocol

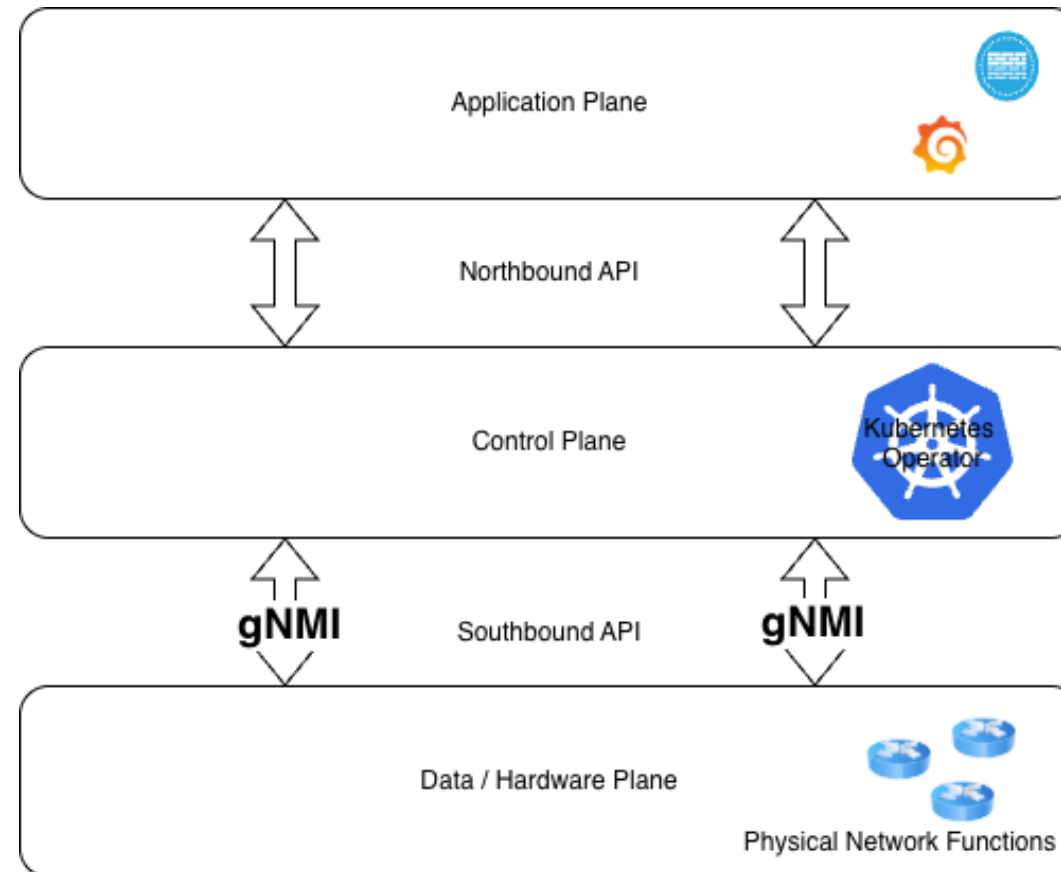
What is gNMI?

- gRPC Network Management Interface
- Developed by Google/OpenConfig
- Uses high-performance binary gRPC

Core Operations

- **Get** (read state)
- **Set** (write config)
- **Subscribe** (stream telemetry)
- **Capability**

My SDN Solution



Approach – The Prototype

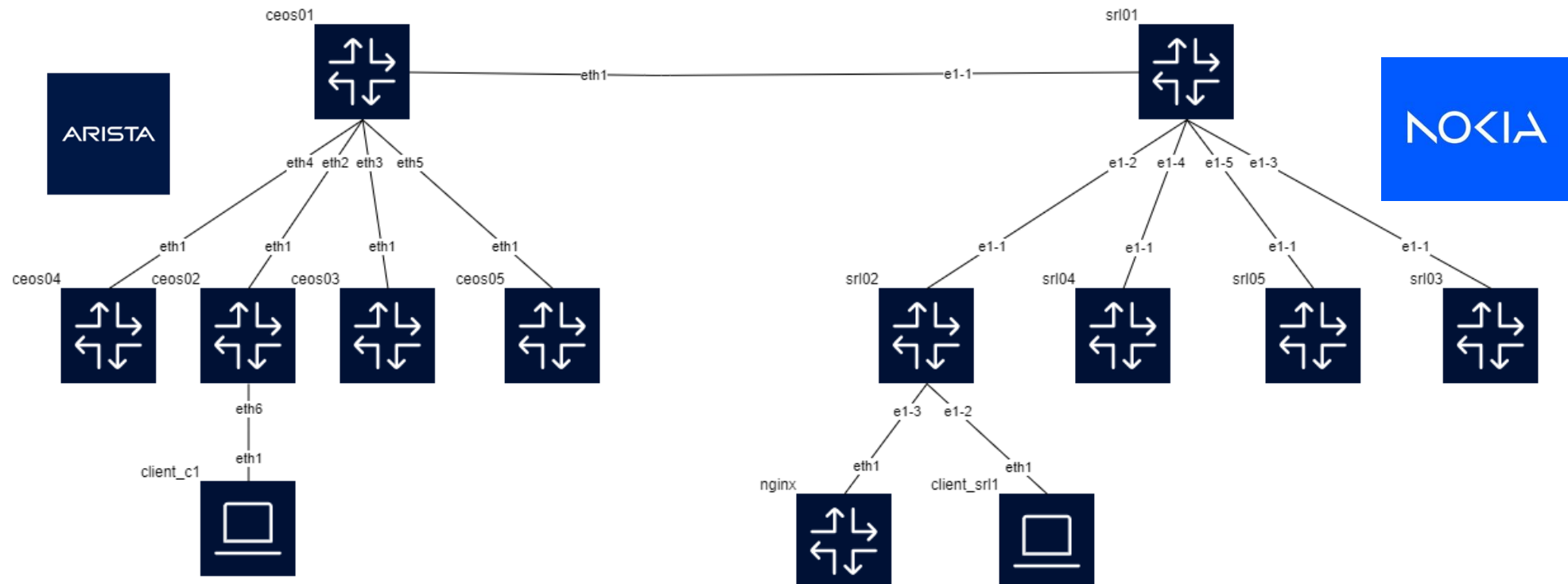


- Prototype for physical device management
- gNMI to configure and connect to the device
 - Get the device name (Read)
 - Set configuration values (Write)
- Continuous connection monitoring

Approach – Test Environment

- Built with Containerlab
- Mixed-vendor gNMI support

- 5x Arista cEOS routers
- 5x Nokia SR Linux (srl) routers



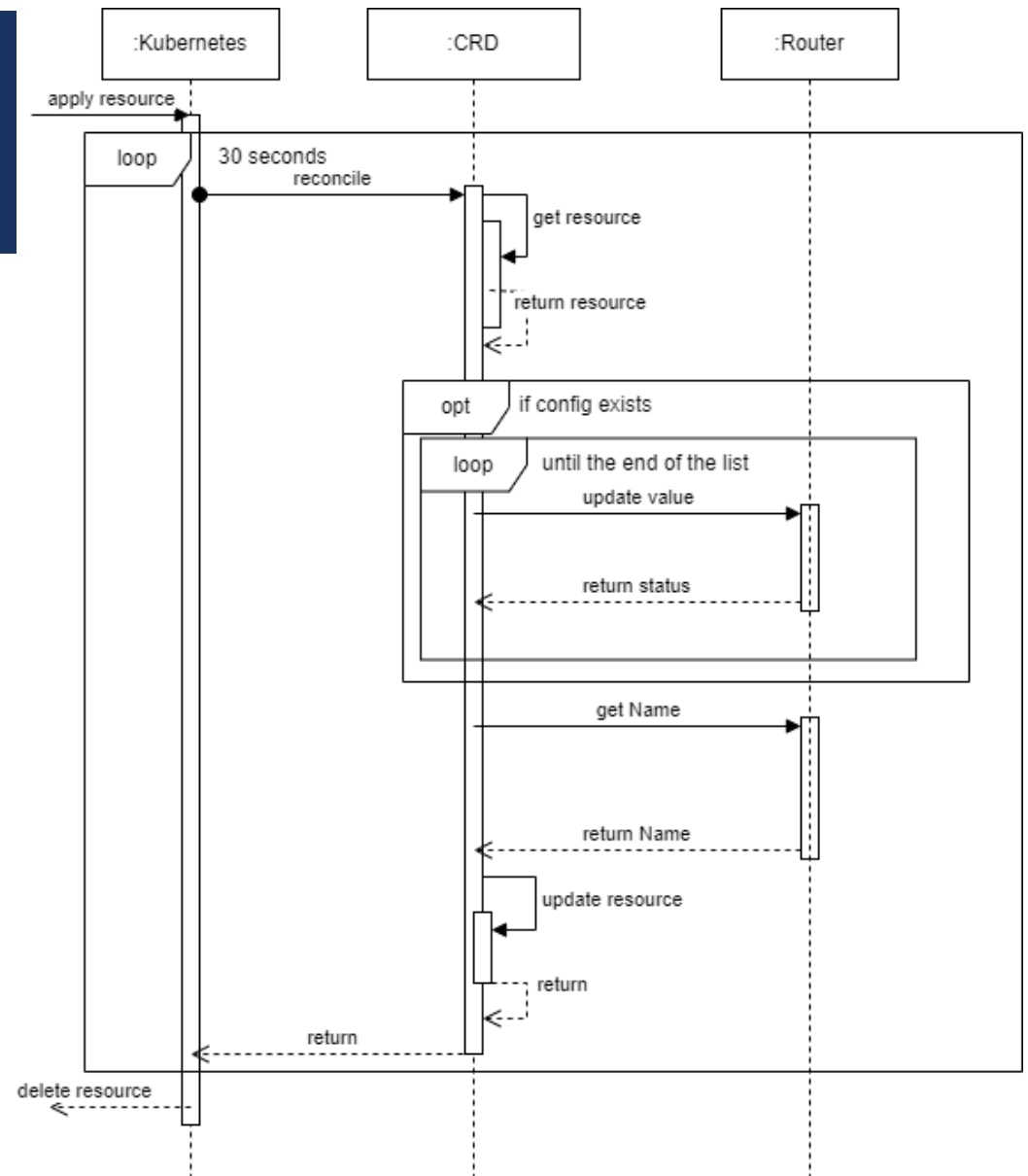
Implementation – The Custom API

```
apiVersion: hardware.siticom.de/v1
kind: Router
metadata:
  labels:
    app.kubernetes.io/name: router-crd
    app.kubernetes.io/managed-by: kustomize
  name: srlinux-01
spec:
  routertype: SRLinux
  ip: router.vhost.internal.siticom.de
  port: 57400
  username: admin
  password: NokiaSrl1!
  config:
    - path: /system/name
      data: '{"host-name": "srlo1-test"}'
    - path: /interface[name=ethernet-1/1]
      data: '{"admin-state": "enable"}'
```

NAME	IP	PORT	ROUTERNAME	ROUTERTYPE
ceos-01	router.vhost.internal.siticom.de	6030	ceos01	
	CONNECTED	AGE		
	CEOS	Connected	11d	
ceos-02	router.vhost.internal.siticom.de	6031	ceos02	
	CONNECTED	AGE		
	CEOS	Connected	11d	
srlinux-01	router.vhost.internal.siticom.de	57400	srlo1	
	SRLinux	Connected	11d	
srlinux-02	router.vhost.internal.siticom.de	57401	srlo2	
	SRLinux	Connected	11d	

Implementation – The Controller

- Fetch resource
- Update Configurations (if any)
- Validate Connection (Get Name)
- Update Kubernetes State



Capabilities



Reading configuration data

- Generalized approach via gNMI Get-function
- Retrieves data from any accessible path (tested via device name)
- Proven on Nokia SR Linux and Arista cEOS



Updating and writing configuration data

- Generalized approach via gNMI Set-function
- Configure zero, one, or multiple values simultaneously
- Proven on Nokia SR Linux and Arista cEOS

Evaluation – The Ansible Baseline

The Setup

- Standard configuration management tool (Industry Baseline)
- Executed via YAML Playbooks and Inventories

The Execution

- Connects to the Containerlab host via SSH
- Utilizes the gnmic CLI client for Get/Set operations
- Parity with the Operator's capabilities

Evaluation - Usability

Criteria	Ansible	CRD
Getting started	Extensive documentation and tutorials	Requires Kubernetes knowledge, but native feel for app teams.
Change config	Playbooks using mixed YAML/JSON	Pure YAML manifests (highly readable).
Integration (Other Environments)	Highly flexible (runs via CLI, GitLab Pipelines)	Native GitOps integration (e.g., FluxCD).

Complexity

Criteria	Ansible	CRD
Device Knowledge & Access	Multiple protocols supported (SSH, SNMP, etc.)	Exclusively gNMI
Adjusting output	Straightforward (edit YAML playbook)	Complex (Requires Go, Kubebuilder, and Redeployment)
Adding devices	Easy (Extensive community modules)	Depends on gNMI availability
Resources & Setup	Minimal (UNIX & Python)	Requires k8s cluster (+1 CPU)

Adjusting output - Ansible

```
1 - { host: clab-kind-srl01:57400, username: admin, password: NokiaSrl1!, type: srl,  
2   path: [/system/name/host-name] }
```

```
1 {  
2   "source": "clab-kind-srl01:57400",  
3   "time": "2025-02-10T09:58:27.541046266Z",  
4   "timestamp": 1739181507541046266,  
5   "updates": [  
6     {  
7       "Path": "srl_nokia-system:system/srl_nokia-system-name:name/host-name",  
8       "values": {  
9         "srl_nokia-system:system/srl_nokia-system-name:name/host-name": "srl01"  
10      }  
11    }  
12  ]  
13 }  
14
```

```
1 - { host: clab-kind-srl01:57400, username: admin, password: NokiaSrl1!, type: srl,  
2   path: [/system/name/host-name, /system/information/version] }  
  
1 {  
2   "source": "clab-kind-srl01:57400",  
3   "time": "2025-02-10T09:58:27.975571489Z",  
4   "timestamp": 1739181507975571489,  
5   "updates": [  
6     {  
7       "Path": "srl_nokia-system:system/srl_nokia-system-name:name/host-name",  
8       "values": {  
9         "srl_nokia-system:system/srl_nokia-system-name:name/host-name": "srl01"  
10      }  
11    }  
12  ],  
13 },  
14 {  
15   "source": "clab-kind-srl01:57400",  
16   "time": "2025-02-10T09:58:27.976128677Z",  
17   "timestamp": 1739181507976128677,  
18   "updates": [  
19     {  
20       "Path": "srl_nokia-system:system/srl_nokia-system-info:information/version",  
21       "values": {  
22         "srl_nokia-system:system/srl_nokia-system-info:information/version": "v24.7.1-330-g38f"  
23      }  
24    }  
25  ]  
26 }
```

Adjusting output - CRD

```
1 // router_types.go
2 // RouterState is to get the values from the Router and present them
3 type RouterState struct {
4     // Name of the Router
5     // +optional
6     RouterName string `json:"routername,omitempty" yaml:"routername,omitempty" protobuf:"bytes,1,opt,name=routername"`
7
8     // Connection State to the router
9     // +optional
10    Connected string `json:"connected,omitempty" yaml:"connected,omitempty" protobuf:"bytes,1,opt,name=connected"`
11
12    // Connection State to the router
13    // +optional
14    Version string `json:"version,omitempty" yaml:"version,omitempty" protobuf:"bytes,1,opt,name=version"`
15 }
```

```
1 // router_types.go
2
3 // +kubebuilder:object:root=true
4 // +kubebuilder:subresource:status
5 // +kubebuilder:printcolumn:name="IP",type="string",JSONPath=".spec.ip"
6 // +kubebuilder:printcolumn:name="Port",type="integer",JSONPath=".spec.port"
7 // +kubebuilder:printcolumn:name="RouterName",type="string",JSONPath=".state.routername"
8 // +kubebuilder:printcolumn:name="RouterType",type="string",JSONPath=".spec.routertype"
9 // +kubebuilder:printcolumn:name="Connected",type="string",JSONPath=".state.connected"
10 // +kubebuilder:printcolumn:name="Version",type="string",JSONPath=".state.version"
11 // +kubebuilder:printcolumn:name="Age",type="date",JSONPath=".metadata.creationTimestamp"
12 // Router is the Schema for the routers API
13 type Router struct {
14     metav1.TypeMeta `json:",inline"`
15     metav1.ObjectMeta `json:"metadata,omitempty" protobuf:"bytes,1,opt,name=metadata"`
16
17     Spec RouterSpec `json:"spec,omitempty" protobuf:"bytes,2,opt,name=spec"`
18     Status RouterStatus `json:"status,omitempty" protobuf:"bytes,3,opt,name=status"`
19     State RouterState `json:"state,omitempty" protobuf:"bytes,4,opt,name=state"`
20 }
```

```
1 // router_controller.go
2
3 switch router.Spec.RouterType {
4     case RouterType.SRLinux:
5         rname, router.State.Connected = srlinux.GetSRLinux(router.Spec.IP, router.Spec.Port,
6             router.Spec.Username, router.Spec.Password, false, true, false, "/system/name/host-name")
7         router.State.Version, router.State.Connected = srlinux.GetSRLinux(router.Spec.IP, router.Spec.Port,
8             router.Spec.Username, router.Spec.Password, false, true, false, "/system/information/version")
9     case RouterType.CEOS:
10         rname, router.State.Connected = ceos.GetCE
```

NAME	IP	PORT	ROUTERNAME	ROUTERTYPE	CONNECTED	VERSION	AGE
ceos-01	10.120.216.48	6030	ceos01	CEOS	Connected		24d
ceos-02	10.120.216.48	6031	ceos02	CEOS	Connected		24d
srlinux-01	10.120.216.48	57400	sr101	SRLinux	Connected	v24.7.1-330-g38f237abfe	24d
srlinux-02	10.120.216.48	57401	sr102	SRLinux	Connected	v24.7.1-330-g38f237abfe	24d
srlinux-03	10.120.216.48	57402	sr103	SRLinux	Connected	v24.7.1-330-g38f237abfe	24d

Features

Criteria	Ansible	CRD
Backup / Recovery	Possible / Hard	No
Monitoring	Possible with AWX	Monitors the connection to the device

Conclusion

- Kubernetes for Network Automation
- Built a custom operator based on gNMI
- Successfully validated Get, Set, and continuous monitoring capabilities
- Cannot *yet* fully leverage all Kubernetes advantages
- Proves Kubernetes is viable for both Virtual (VNF) and Physical (PNF) network automation

Future Work

Advancing gNMI Capabilities

- Implement the gNMI *Subscribe* function.
- Enable real-time telemetry streaming directly to the cluster.

Expanding Protocol Support

- Integrate NETCONF, RESTCONF.
- Extend operator compatibility to legacy network devices.

Dynamic Configurations

- Move away from hard-coded Go structs.
- Develop dynamic path discovery for easier output adjustments.

Deepening Kubernetes Integration

- Implement full configuration state backups.
- Enable automated rollback and recovery features.

Thank You & Questions

- Kubernetes for Network Automation
- Built a custom operator based on gNMI
- Successfully validated Get, Set, and continuous monitoring capabilities
- Cannot *yet* fully leverage all Kubernetes advantages
- Proves Kubernetes is viable for both Virtual (VNF) and Physical (PNF) network automation

Sources

- Agenda by: [Slidesgo](#) and [Freepik](#)
- Kubernetes Operator Components: https://github.com/cncf/tag-app-delivery/blob/163962c4b1cd70d085107fc579e3e04c2e14d59c/operator-wg/whitepaper/Operator-WhitePaper_v1-0.md#operator-components-in-kubernetes
- SDN Architecture: <https://telsoc.org/journal/ajtde-v3-n4/a28>
- Sources of the thesis



Backup



Why these NOS?

Device	GNMI	NETCONF	JSON-RPC	SNMP	CLI
Nokia SR Linux	✓	✗	✓	✓	✓
Arista cEOS	✓	✓	✗	✗	✓
Juniper cRPD	✗	✓	✗	✗	✓
Cisco XRd	✓	✓	✗	✗	✓
Cumulus VX	✓	✗	✗	✗	✗
SONiC	✗	✗	✗	✗	✓
RARE freeRtr	✗	✗	✗	✗	✓

Mapping ETSI MANO to Kubernetes

ETSI MANO	K8s Native	Extended Ecosystem
VIM (Infrastructure Manager)	Control Plane, Kubelet, Pods, CNI (Multus), PV/PVCs	KubeVirt (for legacy VNFs)
VNFM (Lifecycle Manager)	Deployments, StatefulSets, ReplicaSets	CRDs & Custom Operators, Helm
NFVO (Global Orchestrator)	K8s API (Single Cluster scope)	GitOps (FluxCD/ArgoCD), Crossplane, Cluster API

Limitations

Kubernetes limitations

- Max. 300.000 containers
- Max 5.000 nodes

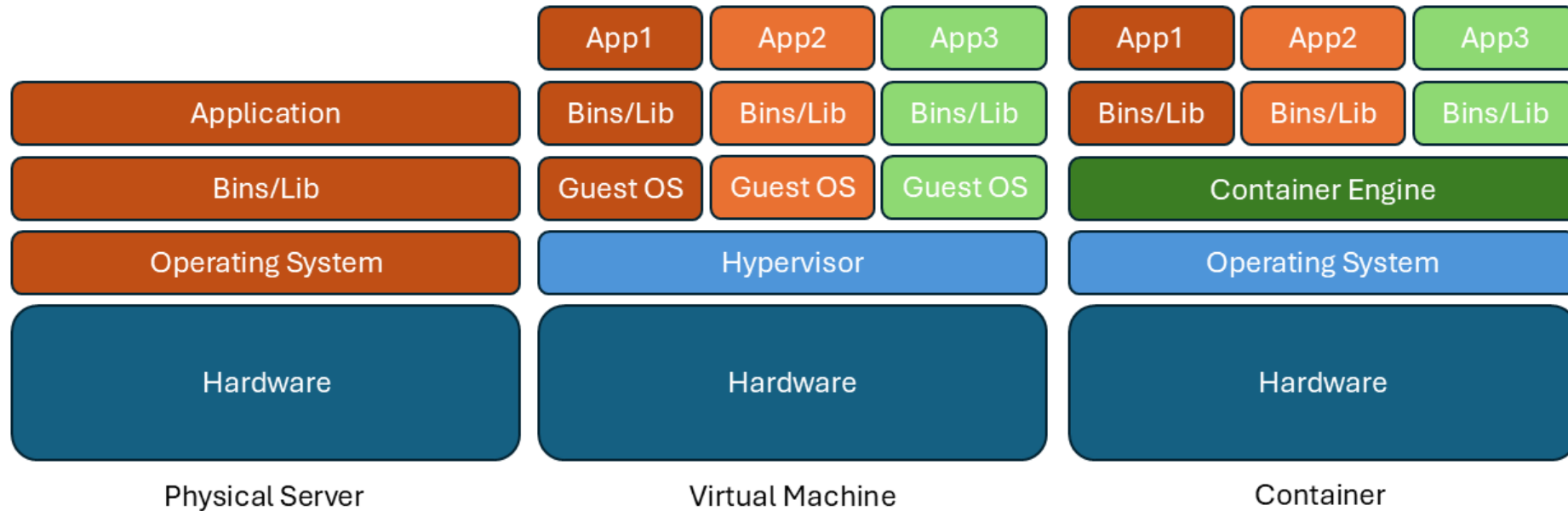
Etcd

- Default request size: 1.5 MB
- Default storage size 2 GB

Operator limitations

- Hard coded get request
- Can't integrate Kubernetes resources (ConfigMaps, Secrets)

Basics - Virtualization



What actually is a Container?

Namespaces (Isolation)

- *Controls what a process can SEE.*
- PID (Process ID)
- Network Stack & Ips
- Filesystem

Control Groups / cgroups

- *Controls what a process can USE.*
- CPU Limits
- Memory Limits (OOM-Killer)
- Network & Disk I/O

Virtual File System

- *Controls how the image is BUILT.*
- e.g., OverlayFS
- Shared Read-Only Base Layers
- Ephemeral Read-Write Top Layer